

Efficient Visual Question Answering under a 3B Parameter Budget

Yookyung Youn | Korea University

Technical report | SCPC 2025 AI Challenge, 1st Place

Abstract

The SCPC 2025 AI Challenge required multiple-choice visual question answering with fewer than three billion parameters loaded for inference. I independently designed and implemented a solution by compressing InstructBLIP-Flan-T5-XL, stabilizing the compressed model through knowledge distillation, and adding custom LoRA adapters. This report develops the reasoning behind that solution: choosing an architecture whose components could be investigated separately, testing where compression was affordable, revising activation-based assumptions after non-monotonic masking results, and distinguishing recovery from further task adaptation. The final model contains 2,986,746,208 parameters, approximately 25.8% fewer than the original. Reported public weighted accuracy increased from 82.83% after pruning to 85.73% after distillation and 86.59% after LoRA. The solution received first place in the AI Challenge. Alongside these results, I discuss qualitative language-generation degradation and the limits of task accuracy as evidence of retained capability. The report presents the design rationale, implementation, experimental findings, and limitations of a solution built on established methods.

1. Setting and Independent Development

1.1 A task and constraints, with an open solution strategy

Each example contains an image, a question, and candidate answers. The objective is to select an answer using visual information and relevant knowledge. I worked as an individual undergraduate participant, taking responsibility for model and dataset selection, compression design, training, evaluation, implementation, and final submission. The central question was which eligible starting point could become a competitive, compliant system within the resources I could actually use.

The rules required fewer than 3B total parameters across all models loaded during inference, including models loaded sequentially. Eligible pretrained weights had to have been publicly released by December 31, 2023; external data had to have been released by June 10, 2025, with licenses meeting the competition requirements. Remote model APIs were prohibited [1]. This was a parameter-count constraint: reduced numerical precision or a small number of activated parameters would not by itself establish compliance.

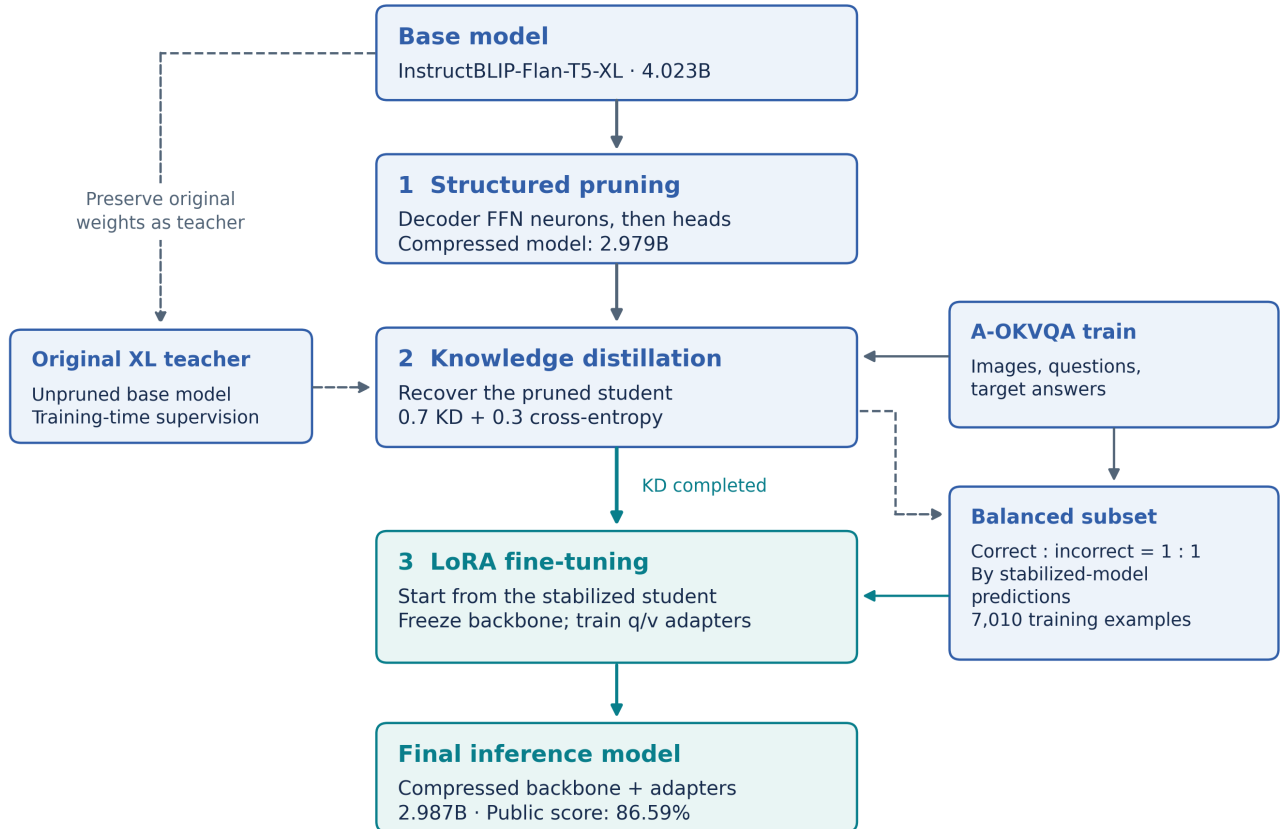
1.2 The effective development window

I concentrated implementation and experiments into approximately two to three weeks because the competition overlapped with end-of-semester commitments. This describes my effective development window, not the official duration of the event. GPU access was paid for personally through on-demand rentals. The cost of a failed run affected which hypotheses I could test and how much evidence I sought before committing to longer training.

These constraints encouraged inexpensive diagnostics, incremental changes, and reuse of pretrained capabilities. They also made stopping decisions consequential. A plausible alternative could still be a poor use of the remaining time if its alignment, training, or debugging costs were difficult to estimate. The choices below should be understood within that setting.

1.3 Independent design and contribution

My contribution is the independent formulation, experimental development, structural implementation, and integration of this solution. I developed several design questions by reasoning about the task and architecture, then used experiments to revise the resulting hypotheses. When investigating FFN importance, I found Wanda's activation-and-weight principle and adapted it to neuron-level pruning under my compute constraints. The system builds on established pruning, distillation, and LoRA methods; the contribution lies in how I selected, adapted, and integrated them to solve the competition task. Section 9 gives credit to these methods and connects the design decisions to related research.



Solid arrows: model or training-data flow. Dashed arrows: teacher or selection guidance.
Inference loads only the final compressed model and adapters; the teacher is not loaded.

Figure 1. Ordered training pipeline: (1) structured pruning, (2) knowledge distillation to stabilize the pruned student, and (3) LoRA fine-tuning of that stabilized student with its backbone frozen. The original model supplies teacher supervision only in stage 2. A-OKVQA training examples support distillation; a subset balanced by the stabilized model's correctness supplies stage 3. Final inference loads the compressed model and trained adapters.

2. Selecting a Model I Could Investigate

2.1 The eligible model pool did not provide an obvious answer

My initial intention was to start with a capable larger model and prune it. The eligible candidates I explored presented difficult trade-offs: some already fit below 3B, some exceeded the limit without a compelling task-performance advantage over smaller alternatives, and others required much more aggressive compression. This made the starting model's suitability for compression as important as its initial task performance.

I also considered assembling separately pretrained components and adapting a more unified architecture. The problem was to find both useful starting capability and an experimental path whose failures I could diagnose. A high initial score alone did not tell me how manageable the compression problem would be.

2.2 Why I deferred assembling separate components

Combining a strong encoder with a separately pretrained language component appeared attractive because it would let me choose the size of each part. My concern was that separately learned representations would not automatically be compatible. A bridge could reconcile tensor dimensions without necessarily preserving the meaning or behavior expected by the receiving component.

I anticipated two costs. First, learning a useful bridge might require more data and GPU time than I could afford. Second, unfreezing the components to force alignment could disturb their useful pretrained behavior and create another recovery problem. At the time I worried that alignment might approach pretraining-scale effort. That was a conservative risk estimate, not a demonstrated lower bound on the required data.

I therefore deferred this route within the competition window. The decision was that uncertain alignment costs made it less predictable than modifying a model whose components already worked together. It was not a finding that component assembly is generally ineffective. Section 9 revisits the data-cost assumption using relevant research.

2.3 Why InstructBLIP-Flan-T5-XL made component-level experiments practical

For some more unified architectures I explored, the effects of pruning were difficult to localize. A change could affect tightly coupled functions, and the resulting losses were irregular. I wanted to ask more focused questions about where information was transformed and which component had changed.

InstructBLIP-Flan-T5-XL includes an image encoder, a Q-Former, and a T5 language model with separate encoder and decoder components [3]. That decomposition created more opportunities to intervene in one part and observe the consequence. The Q-Former connects visual features to the language model; it is distinct from the connection between the T5 encoder and decoder.

The choice combined existing task capability with experimental tractability. I accepted a substantial compression requirement because the architecture gave me a clearer way to investigate it. The baseline contains 4,022,969,088 parameters and achieves 79.00% accuracy on a 1,000-example A-OKVQA validation subset. That is a local baseline, not a competition public score.

3. From Architectural Intuition to Pruning Experiments

3.1 Why I expected some interventions to be more costly

Although the final system prunes decoder FFNs and attention heads, the initial candidate set included the image encoder, language encoder, Q-Former, decoder representations, FFNs, and attention heads.

My first hypothesis concerned information flow. If an early transformation corrupts a representation used by many subsequent computations, later components must operate on that altered input. I expected more opportunities for error propagation than from a localized change near the output. This was the intuition behind my informal description of a snowball effect. It suggested where to investigate sensitivity; it did not establish a monotonic relationship between depth and pruning tolerance.

My second concern was the interface between modalities. The Q-Former appeared especially consequential because it mediates visual information entering the language model. Disrupting that interface could make otherwise capable components less useful together. I therefore wanted evidence of sufficient savings and tolerance before altering it.

Exploratory pruning tests on both the image encoder and language encoder produced severe, irregular performance losses even under relatively small interventions. These observations were consistent with the information-flow concern and made those components unattractive targets under the available budget. I therefore preserved the encoders and visual-language interface and concentrated further experiments on decoder components.

Related research helps explain the importance of this interface. In BLIP-2, omitting the Q-Former's first-stage representation learning substantially reduces zero-shot VQA performance, including with Flan-T5-XL [9]. ECoFLaP, an ICLR 2024 pruning study, retains the Q-Former while compressing visual and language backbones because it accounts for only about 5% of parameters in the studied models [14]. Together, these findings support a practical rationale for preserving learned alignment when potential savings are limited. They provide evidence about alignment learning and compression allocation, rather than a direct sensitivity measurement for pruning my model's Q-Former. Both studies predate the competition and are discussed here as related evidence.

3.2 Choosing the data before choosing the score

I wanted to retain computation useful for the competition task. That made calibration data part of the pruning design. I selected A-OKVQA for its combination of visual questions, world knowledge, and a multiple-choice formulation [4]. I also considered OK-VQA, but the answer-selection format was an additional reason to prioritize A-OKVQA.

The question was: when the model performs a similar task, which parts of its computation participate, and how does pruning those components affect task performance? I first tested candidate interventions through temporary masking before committing to structural removal. Activation was attractive because it was observable with modest additional computation. It supplied an initial signal, rather than a direct measurement of semantic importance.

3.3 Why activation clustering changed my assumptions

I investigated hidden-state masking using activation statistics and clustering. I began with relatively low-activation groups, expecting smaller performance losses. Many values were similarly small, however, so a coarse grouping could combine dimensions with very different effects. I subdivided groups to obtain finer candidates within the compute budget.

Masking did not produce a clean, monotonic relationship between activation magnitude and damage. Some low-activation groups caused substantial losses; other combinations changed little, and some even unexpectedly improved performance. Low activation on a limited sample could not establish that a dimension was dispensable.

I considered whether some dimensions supported useful background information without becoming strongly active in those examples. I also considered interactions: removing one dimension could have a different effect depending on which others remained. I imagined complementary or mutually constraining groups. Those were hypotheses motivating further tests, not evidence identifying specific neurons as storing background knowledge or demonstrating a particular mechanism.

I then used a greedy process, evaluating candidate masking choices in the context of choices already made. Activation continued to propose candidates, and I tried additional combinations to avoid relying entirely on an initial ranking. The change in reasoning was to treat usefulness as conditional on the retained model. The search was neither exhaustive nor guaranteed to find a global optimum, but it directly tested the assumption that a static ranking had been missing.

3.4 Why I removed hidden-state masking from the final design

The masking experiments showed a region of relatively stable performance followed by sharp losses under more extensive intervention. Pursuing further removal therefore became difficult to justify: the additional parameter savings were small relative to the risk of disrupting task performance.

Initially I investigated FFNs with the hidden-state intervention already present because I expected good removal combinations to depend on the current architecture. Once FFN compression supplied most of the useful savings, I reconsidered whether hidden-state removal still earned its cost. I chose the simpler final path based on FFN neurons and attention heads.

This was a substantive design revision. An idea could be worth investigating without belonging in the submission: these experiments exposed non-monotonic sensitivity and the importance of combinations. In the final architecture, the language model's input/output hidden width remains 2,048. Earlier hidden-state masking is an exploratory branch, not an additional final pruning stage.

4. Allocating the Parameter Savings

4.1 Why FFNs came before attention heads

I compared removal units by both their parameter savings and the behavior they might disturb. My intuition was that attention heads could capture complementary relationships in the input, so removing many heads at once risked losing useful diversity. This motivated a search for another component that could supply a large share of the required savings before reducing the number of heads.

Decoder FFN intermediate neurons offered finer candidates with substantial aggregate savings. My accounting suggested a staged allocation: obtain a large part of the reduction from FFNs, then use head removal to meet the remaining budget. This ordering emerged from parameter accounting and experiments; the final pair of components had not been the only candidates considered.

4.2 From an activation idea to a Wanda-inspired criterion

Activation alone had proved insufficient. I wanted to incorporate how strongly a neuron's activity could affect subsequent computation. The output projection provided an inexpensive additional observation. A neuron with both small observed activation and small outgoing influence seemed a more plausible candidate than one assessed through either quantity alone.

When looking for a formal criterion, I encountered Wanda's weight-and-activation principle [5]. It supported the direction I had been considering, and I adapted it to the neuron groups and measurements I could afford. This is where a specific literature method entered the FFN design. I do not claim an independently proven formula or an exact implementation of Wanda's original elementwise criterion.

For intermediate neuron j in decoder layer l , the importance score is:

$$\text{score}(l, j) = \text{mean}(\text{abs}(\text{gated_activation}(l, j))) * \text{L2_norm}(\text{wo}(l)[:, j])$$

The mean uses activations collected by the calibration routine during generation on the first 250 A-OKVQA training examples. It is a proxy for contribution on sampled inputs, not a bound on information loss. Its value is that it narrows an otherwise costly search while remaining cheap to compute.

4.3 What the final FFN operation actually removes

The decoder originally has 24 gated FFNs, each with intermediate width 5,120 and input/output width 2,048. I globally ranked intermediate neurons across decoder layers and selected the lowest-scoring 95% for removal. Candidate masking suppressed entire columns of the output projection. Final compaction removed the corresponding rows of both input projections, wi_0 and wi_1 , and columns of wo , rebuilding smaller dense layers.

This is neuron-level structured pruning. The FFN intermediate widths shrink while the model-facing hidden width stays fixed. The 95% rate applies across decoder FFNs in aggregate, not separately to every layer or to the whole model. The recorded FFN count changes from 754,974,720 to 37,748,736 parameters.

Layer-level inspection and comparisons of removal rates showed non-monotonic accuracy changes as pruning increased. I therefore judged candidate rates by both task performance and the resulting parameter count. Among the tested settings, 95% aggregate FFN-neuron removal provided a useful balance and supplied most of the required reduction. The global ranking allowed retained widths to vary across layers rather than imposing the same removal fraction everywhere; it did not explicitly equalize performance damage across layers.

4.4 From a dependency hypothesis to head-similarity analysis

After substantial FFN removal, I expected some attention computations to become less useful or partly redundant. I considered whether a head might lose utility when associated FFN computation was removed. This was a reason to investigate dependencies, not an implemented mapping between specific heads and FFN neurons.

The actual signal was attention-map similarity. If two heads behaved similarly on inspected inputs, I hypothesized that removing one could preserve more useful behavior than removing heads with different patterns. Similarity narrowed the candidate set; task evaluation still had to decide. Similar maps do not imply interchangeable value projections or identical downstream contributions.

The head-analysis procedure computes cosine similarities on the original unpruned model, using 500 A-OKVQA training images and a generic image-description prompt. Greedy candidate selection and approximately 60 removal configurations were evaluated on a 100-example validation subset. Head-only tests screened candidates before combined evaluation with FFN pruning. This separated inexpensive similarity-based proposal from task-based selection in the compressed system. The criterion uses attention-map similarity; it does not compute explicit head-to-FFN associations.

4.5 The retained structure

The zero-based removed indices are [1, 3, 4, 8, 10, 11, 12, 14, 17, 21, 24, 25, 26]. The same set is applied to decoder self-attention and cross-attention across 24 layers. Each module retains 19 of 32 heads.

Compaction removes the associated rows of query/key/value projections and columns of the output projection. At 64 dimensions per head, internal width changes from 2,048 to 1,216 while input/output hidden widths remain fixed. The recorded attention count changes from 805,306,368 to 478,150,656 parameters.

The resulting model has 2,978,586,976 parameters. Reported A-OKVQA validation accuracy falls from 79.00% to 76.60% on 1,000 examples, while separate public weighted accuracy is 82.83%. Meeting the size constraint left a concrete next problem: recover useful behavior in the compliant architecture.

5. Knowledge Distillation as Stabilization

5.1 Matching the teacher to the recovery objective

Pruning solved the size constraint but reduced task accuracy. I approached the next stage as recovery: the student retained the original weights in a smaller architecture, so I wanted to restore useful behavior that the original model had already supported. Distillation could supply that supervision without increasing the student's inference-time parameter count.

This objective shaped teacher selection. The original unpruned InstructBLIP-Flan-T5-XL provided a reference closely related to the student's starting point. A larger teacher could offer stronger predictions, but its behavior might be harder for the compressed student to reproduce with limited data and GPU time. I therefore evaluated teacher suitability by recovery and transfer, rather than size alone.

A larger teacher from the InstructBLIP-Flan-T5 family improved fit to the A-OKVQA training data, but the original XL teacher produced better A-OKVQA validation and public-evaluation performance. The stronger fit did not translate into better transfer. I selected the original teacher because it better served the immediate objective: stabilizing the pruned model before attempting further adaptation.

5.2 Choosing supervision the compressed student could use efficiently

I considered hidden-state matching because pruning had altered the student's internal computation. However, reproducing detailed teacher representations could require capacity that the student no longer possessed. Keeping the necessary intermediate activations also increased GPU memory demands. Hidden-state experiments encountered memory limits and provided less useful recovery than answer-focused supervision, leading me to prioritize output targets. Capacity mismatch motivated the comparison; it was not isolated as the sole explanation for the observed difference.

The retained procedure matches answer-token distributions and trains on target answer text. This places supervision at the behavior the competition directly evaluates, while allowing the smaller model to arrive at that behavior through its own internal representations. It made recovery more practical within the available training budget.

I also tested explanation-based training because pruning had degraded language generation beyond answer selection. Teacher-generated rationales offered a way to restore some of that behavior. They improved language generation qualitatively, but their additional answer-selection benefit was too small relative to GPU cost to justify inclusion in the final pipeline. I therefore retained answer-focused distillation for stabilization. In the rationale prototypes, auxiliary quality metrics computed outside the differentiable graph served as evaluation signals rather than additional gradient-based training objectives.

5.3 Stabilization objective and training

The selected stabilization procedure combines soft-target distillation with hard-label cross-entropy [6]. Soft targets convey the teacher's output distribution; hard labels anchor the update to the target answers:

$$L = 0.7 * L_{KD} + 0.3 * L_{CE}$$

$$L_{KD} = T^2 * KL(\text{softmax}(\text{teacher_logits} / T) || \text{softmax}(\text{student_logits} / T)); T = 3$$

The implementation uses `kl_div` with `reduction='batchmean'`. Teacher and student receive images, questions, and target answer sequences during training. Hard labels are answer text, not merely A/B/C/D. This is token-distribution matching under teacher forcing. The KL call operates on the logits tensor without a separate padding mask.

Stabilization setting	Value
Teacher and student	Original InstructBLIP-Flan-T5-XL in evaluation mode; structurally pruned student
Data and internal split	A-OKVQA train, 17,056 examples; 90% training / 10% validation, seed 42
Temperature and mixture	$T = 3$; $0.7 \text{ KD} + 0.3 \text{ cross-entropy}$
Epochs and learning rate	3; $5e-6$
Batch size and accumulation	1; 4 steps
Schedule and clipping	Linear schedule with 10% warmup; gradient norm clipped at 1.0
Checkpoint selection	Lowest internal validation cross-entropy, evaluated every 1,000 optimizer steps

A 0.65/0.35 mixture was also explored during development. The table describes the final stabilization implementation; the available results do not provide a matched comparison of loss mixtures.

Distillation retains the pruned architecture. The reported KD-stage public score is 85.73%, an increase of 2.90 percentage points from the pruned checkpoint. These stage results establish the role of KD in the submitted pipeline, while the loss-weight alternatives were not isolated in a controlled ablation. With stabilization complete and parameter headroom still available, I moved to a separate stage of task adaptation. The teacher is used only for distillation and is absent from final inference.

6. LoRA as Controlled Task Adaptation

6.1 Using the remaining budget without disturbing the recovered backbone

After stabilization, the model still had room below 3B. I could therefore add limited trainable capacity to improve task performance. LoRA made that addition controllable through adapter rank and the choice of target projections [7]. Its purpose in this pipeline was further adaptation after KD had recovered useful behavior.

The amount of task data shaped how I trained the additional capacity. Full-model fine-tuning would update broadly pretrained weights using the much smaller A-OKVQA dataset, risking over-specialization and loss of behavior restored by KD. Full-update trials showed degradation consistent with that concern. I consequently froze the stabilized backbone and trained only the adapters, restricting the update while preserving the recovered base weights.

6.2 Making adapters fit the structurally compressed model

Pruning had changed attention projection dimensions, and the library adapter route I initially tried produced errors. The implementation addresses this by wrapping the actual compressed linear layers and deriving adapter dimensions from their `in_features` and `out_features`. This makes the adapter shape follow the modified model rather than depend on the original projection widths.

The wrapper adds two trainable low-rank matrices to the frozen linear transformation, scaled by α/rank . The selected rank is 16, α is 32, and adapter dropout is 0.1. Query and value projections in T5 encoder self-attention and decoder self-attention/cross-attention are wrapped, for 144 adapted projections. Only the adapter parameters train. This custom implementation makes an established method compatible with the compressed architecture.

The adapters add 8,159,232 parameters, bringing the final count to 2,986,746,208 and leaving 13,253,792 below the 3B boundary. Checking this count tied the adaptation design directly to the inference constraint: useful extra capacity had to fit inside the remaining allowance.

6.3 Balancing correction with preservation

Freezing the backbone controlled which weights could change, but the training examples still determined the direction of adaptation. With limited task data, using only already-correct examples would provide little direct pressure to fix failures and could overemphasize familiar cases. Using only incorrect examples would concentrate the update on the stabilized model's particular weaknesses and remove examples of behavior worth preserving.

I therefore treated the two groups as serving different purposes: incorrect examples supplied corrective targets, while correct examples continued to exercise successful behavior. I partitioned A-OKVQA training examples by the stabilized model's predictions and compared sampling ratios. A 1:1 mixture gave the most useful balance in A-OKVQA and public-score comparisons, so I selected it for LoRA training.

The implementation reads saved incorrect-question identifiers, samples equal numbers from the two groups with seed 42, and shuffles the resulting 7,010 examples. Equal sampling deliberately reweights the task data toward this preservation-and-correction objective. Its selection reflects the comparisons in this project, rather than a claim that equal proportions are optimal for every model or VQA distribution.

6.4 Using transfer behavior to judge training duration

Initial A-OKVQA improvements were accompanied by public-score improvements, supporting continued adaptation. With longer training, however, A-OKVQA accuracy continued to rise while the public score fell. The divergence changed the stopping decision: better performance on the adaptation domain was no longer sufficient evidence of a better competition model.

Training used external A-OKVQA examples; competition evaluation data were not used for gradient updates. The diverging trends were consistent with increasing specialization to the adaptation data. I therefore favored the earlier model with better transfer instead of extending training solely to improve A-OKVQA accuracy. The final LoRA implementation reloads the step-1,000 checkpoint; this is the selected checkpoint for the project, not a general stopping threshold.

Final public weighted accuracy is 86.59%, an additional 0.86 percentage points after stabilization. This completes the intended sequence: pruning supplies the parameter savings, KD recovers behavior in the smaller architecture, and LoRA uses the remaining allowance for controlled task adaptation. Public feedback informed these development choices, so the public score is not an untouched generalization test.

7. Quantitative Results and Evaluation Boundaries

7.1 Stage-wise results

Stage	Parameter count	A-OKVQA validation accuracy	Public weighted accuracy
Original model	4,022,969,088	79.00%	-
Structured pruning	2,978,586,976	76.60%	82.83%
Knowledge distillation	Same pruned architecture	-	85.73%
Final model with LoRA	2,986,746,208	-	86.59%

The A-OKVQA column reports the 1,000-example validation comparison, with a 2.40 percentage-point decrease after pruning. A dash means no verified value is supplied for that metric and stage. Public weighted accuracy uses a different dataset and metric, so comparisons must stay within a column. In particular, 86.59% cannot be compared with the original model's 79.00% as a gain on the same benchmark.

The final model is 25.76% smaller than the original, rounded to 25.8% in overview text. Before adapters, the pruned model is 25.96% smaller. The public gain over the pruned stage is 3.76 percentage points. These cumulative checkpoints do not isolate the causal contributions of KD, LoRA, and balanced sampling.

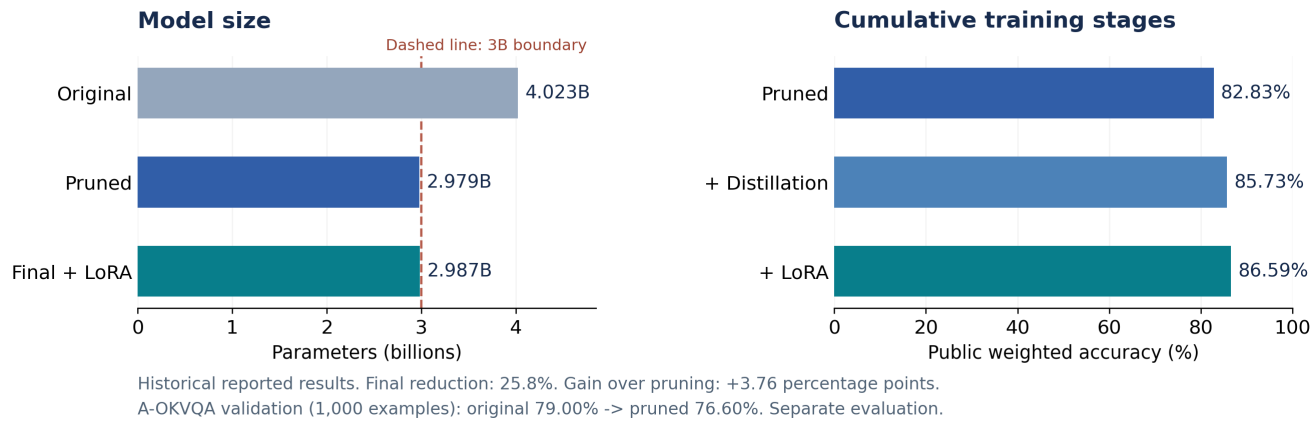


Figure 2. Parameter counts and public scores at cumulative pipeline stages. The local A-OKVQA comparison is stated separately because it uses a different dataset and metric.

7.2 Different subsets answered different questions

Use	Data	Role and limitation
Original/pruned comparison	A-OKVQA validation, 1,000 examples	Local baseline comparison
FFN calibration	First 250 A-OKVQA training examples	Importance signal; no random-sampling guarantee
Larger masking diagnostic	2,000 A-OKVQA training examples	Development sensitivity analysis
Head similarity	500 A-OKVQA training images, generic description prompt	Candidate signal with a prompt mismatch to multiple-choice VQA
Head-removal search	100 A-OKVQA validation examples	Small development subset; roughly 60 configurations
KD internal validation	10% of the 17,056-example A-OKVQA training split	Checkpoint selection by cross-entropy
LoRA training	7,010 A-OKVQA training examples	Balanced by preceding-model correctness

The subsets can overlap and should not be added as a count of unique examples. KD's internal holdout is distinct from the official validation split and is not guaranteed to remain held out from subsequent LoRA training. A separate 60-example competition diagnostic is not an A-OKVQA measurement and is not inserted into the stage table.

The public score and final award are distinct. Competition rules describe private-score evaluation and final presentation assessment [1]. Samsung's announcement documents the first-place award [2]; the public score alone did not define that outcome.

7.3 Historical resources

The experiment summary lists a 32GB RTX 5090 for analysis and inference, an 80GB A100 for distillation at approximately seven hours, and a 24GB RTX 3090 for LoRA at approximately thirty minutes. These describe particular stages, not total experimentation cost or controlled latency, memory, or throughput measurements. Personally funded rentals and the two-to-three-week effective development window made efficient experiment selection an important part of the design.

8. What Task Accuracy Did Not Capture

8.1 Answer selection and language generation diverged

During development, I observed that a pruned model could still select correct answers while its ability to generate useful natural-language explanations deteriorated. I explored rationale training partly because this loss was visible despite acceptable answer behavior. This changed what I considered a complete evaluation of compression.

This was a qualitative language-generation observation; the project did not include a standardized explanation benchmark or a systematic error taxonomy. It motivated a broader evaluation question without establishing that knowledge remained intact while only its expression was damaged. Fluent explanations would also require separate evaluation of whether they faithfully describe the computation behind an answer.

Nevertheless, the experience exposed a concrete evaluation gap: preserving the metric used for selection did not ensure preservation of other useful behavior. It became a motivation for my interest in trustworthy AI and in measurements that reveal capability changes hidden by strong task scores.

8.2 How this experience informs my research questions

For systems that interact with people or act in the physical world, I want to understand whether efficiency and task-performance improvements preserve the capabilities people need to assess failures and intervene. The competition project did not measure robot safety, explanation faithfulness, or robustness in hazardous situations. It provided the experience from which these questions arose.

The question I would now ask is which capabilities changed, under which inputs, and which measurements would reveal an important loss before deployment. This interest follows from examining a limitation of my own successful system.

8.3 Limits of the experimental conclusions

The process involved one task setting, adaptive search, small subsets, and public feedback. Encoder-pruning, alternative-teacher, sampling-ratio, extended-training, and language-generation comparisons are qualitative development observations. The quantitative table is limited to the reported stage measurements. There are no repeated-seed uncertainty estimates, matched-compute teacher comparisons, complete ratio sweeps, or factorial ablations in this release; some development comparisons changed several factors. The final KD configuration is documented in the implementation, but configuration-to-submission mapping is insufficient for a numerical loss-mixture comparison. No new training or inference was run for this report, and possible overlap with the base model's pretraining data was not evaluated.

The evidence supports a working solution under concrete constraints and a record of how I used affordable observations to choose the next intervention. It does not establish universal superiority of one architecture, pruning order, teacher size, or sampling ratio.

9. Related Work

9.1 Methods used and intellectual credit

The system builds on InstructBLIP [3], A-OKVQA [4], Wanda's activation-and-weight principle [5], knowledge distillation [6], and LoRA [7]. Explaining how I arrived at an activation-and-output-weight question does not replace credit to Wanda. The recovery motivation explains my choice of distillation, rather than claiming that I originated teacher-student learning.

The connections below place the design decisions in a broader research context. They are comparisons with related work, rather than additional methods used in the competition implementation. Papers are grouped by publication timing to distinguish research already available in 2025 from subsequent studies.

9.2 Related research available before the competition

Gromov et al., The Unreasonable Ineffectiveness of the Deeper Layers, ICLR 2025 [8], show that some QA performance can survive layer removal while other evaluations deteriorate. This relates to capability-dependent sensitivity. Their language-model layer-removal setting differs from my InstructBLIP FFN/head compression and does not directly establish my explanation-generation observation.

Li et al., BLIP-2, ICML 2023 [9], demonstrate a Q-Former connecting frozen pretrained visual and language components. Maniparambil et al., Harnessing Frozen Unimodal Encoders for Flexible Multimodal Alignment, CVPR 2025 [10], study efficient alignment through projection modules and frozen encoders. These qualify my concern about pretraining-scale alignment costs: such costs are not necessary in every setting. The latter work primarily evaluates classification and retrieval, and its alignment savings reuse already-trained backbones. It does not guarantee inexpensive assembly of arbitrary generative VQA components.

Shu et al., LLaVA-MoD, ICLR 2025 [11], combine sparse MoE and distillation for smaller multimodal models. MoE remained a direction I wanted to investigate with more resources and eligible starting points. Fewer activated parameters alone would not meet this competition's total-loaded-parameter limit. Publication before the competition also does not establish that released weights met its December 2023 cutoff.

Sung et al., ECoFLaP, ICLR 2024 [14], study multimodal compression with adaptive layer-wise pruning. Their choice to preserve the relatively small Q-Former provides a relevant allocation precedent for the interface-preservation reasoning in Section 3.1. Their weight-pruning procedure differs from this project's structural neuron and head removal.

9.3 Research published after the competition

Emmons et al., A Pragmatic Way to Measure Chain-of-Thought Monitorability, Google DeepMind, October 2025 preprint [12], distinguish whether reasoning is readable and whether it contains the steps needed to reach an answer. This supplies vocabulary for properties beyond correctness. Its proxy metrics concern reasoning traces and do not establish whether my model's monitorability changed.

Wen et al., SlimVLM, Huawei, August 2026 preprint [13], study VLM-specific structural pruning with module-dependent sensitivity and LoRA recovery. This relates to where capacity can be removed and how much performance can be recovered. Different models and criteria prevent treating it as validation of my exact removal rate, component ordering, or upstream-error hypothesis. Both corporate studies are identified as preprints rather than assumed conference publications.

10. Design Decisions and Their Consequences

Constraint or observation	Decision	Consequence for the system
Tight parameter budget and costly experiments	Select an aligned model with separable components	Component-level tests guide compression
Severe losses in exploratory encoder tests	Preserve encoders and the visual-language interface	Concentrate removal on decoder components
Low activation alone poorly predicts masking effects	Refine groups and test conditional combinations	Use task evaluation to revise candidate rankings
Limited extra benefit from hidden-state intervention	Remove it from the final design	Retain the 2,048-dimensional model-facing hidden width
Need substantial savings before removing many heads	Rank FFN neurons with an activation-and-weight score	Remove 95% of decoder FFN neurons in aggregate
Potential overlap in attention behavior	Use attention-map similarity and task evaluation	Retain 19 of 32 heads per targeted module
Pruned model needs recovery	Use the original XL teacher and answer-focused KD	Stabilize the smaller architecture before adaptation
Remaining parameter allowance and limited task data	Add custom LoRA with the backbone frozen	Train adapters without updating recovered base weights
Correction alone may neglect successful behavior	Mix correct and incorrect examples equally	Train on a 7,010-example preservation-and-correction mixture
Longer training improves A-OKVQA but reduces public score	Favor the earlier checkpoint with better transfer	Select the step-1,000 LoRA checkpoint
Answer selection survives despite poorer generation	Identify capability evaluation beyond task accuracy as a research need	Treat language-generation loss as an important limitation

This public release contains the technical report, newly drawn figures, and aggregate results. The report connects each intervention to the problem it addressed, the observations that shaped it, and its role in the final solution. The competition presentation, implementation notebooks, checkpoints, and raw datasets are not distributed; this release documents the project rather than providing an executable reproduction package.

References

1. DACON. [SCPC 2025 AI Challenge: official rules](#). Competition constraints and evaluation procedure.
2. Samsung Research. [Samsung Electronics Unveils Winners of the 11th Samsung Collegiate Programming Challenge](#). Official announcement, 2025.
3. Dai et al. [InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning](#), NeurIPS 2023. [Selected model](#).
4. Schwenk et al. [A-OKVQA: A Benchmark for Visual Question Answering using World Knowledge](#), ECCV 2022. [Dataset repository](#).
5. Sun et al. [A Simple and Effective Pruning Approach for Large Language Models](#), ICLR 2024. [Wanda](#).
6. Hinton, Vinyals, and Dean. [Distilling the Knowledge in a Neural Network](#), 2015. Foundational method reference.
7. Hu et al. [LoRA: Low-Rank Adaptation of Large Language Models](#), ICLR 2022.
8. Gromov et al. [The Unreasonable Ineffectiveness of the Deeper Layers](#), ICLR 2025; preprint first released in 2024.
9. Li et al. [BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models](#), ICML 2023.
10. Maniparambil et al. [Harnessing Frozen Unimodal Encoders for Flexible Multimodal Alignment](#), CVPR 2025; preprint first released in 2024. [Author repository](#).
11. Shu et al. [LLaVA-MoD: Making LLaVA Tiny via MoE-Knowledge Distillation](#), ICLR 2025; preprint first released in 2024.
12. Emmons, Zimmermann, Elson, and Shah. [A Pragmatic Way to Measure Chain-of-Thought Monitorability](#), Google DeepMind, October 2025 preprint.
13. Wen et al. [SlimVLM: Sensitivity-aware Dynamic Structured Pruning with Adaptive Visual Token Selection for Efficient Vision-Language Models](#), Huawei, August 2026 preprint.
14. Sung, Yoon, and Bansal. [ECoFLaP: Efficient Coarse-to-Fine Layer-Wise Pruning for Vision-Language Models](#), ICLR 2024.